Parallel Matrix Multiplication Study

John Nehls

10/11/15

Note: This project may be found on GitHub at https://github.com/JohnNehls/ParallelMatrixMultiplication

Contents

1 Introduction **2**

2 Parallel Matrix Multiplication **2**

3 Results **3**

3.1 Total Matrix Calculation Time 3

3.2 Ratio of Communication Over Calculation Time 4

Appendices **5**

A Serial Code **5**

B OpenMP Code **7**

C MPI Code **9**

D Matrix.h **12**

1 Introduction

This document will serve as a comparison of serial code, OpenMP paralell code, and MPI code that accomplishes the same task: matrix multiplication. We will compare the total run times from the codes as well as the amount of time spent calculating the multiplication to the time communicating the data needed to compute the multiplication in parallel. Then, we will explain why the parallelization schemes scale the way they do.

2 Parallel Matrix Multiplication

The scheme used to distribute the matrix multiplication

$$A \times B = C \tag{1}$$

is the approach that was displayed in the lecture 2. To explain our apploach we figures slightly modified versions of figures shown from the lecture (which appear to have been taken from http://parallelcomp.uw.hu/ch03lev1sec1.html).

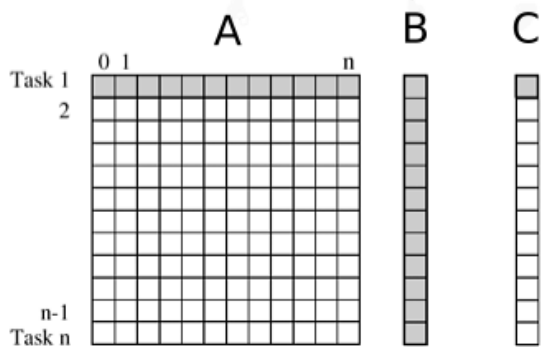

Figure 1: http://parallelcomp.uw.hu/ch03lev1sec1.html

in figure 5 we see how the matrix multplication is split accross the tasks. Task 1 computs the the first element of C from knowing the information in the dark grey: the first row of A and all of matrix B. This is a general way to see the how the computation may be split in to n tasks. In practice often each task will calculate more than one quantitiy. In figure 2 we see how the multiplication of equation 1 is split over 4 tasks.

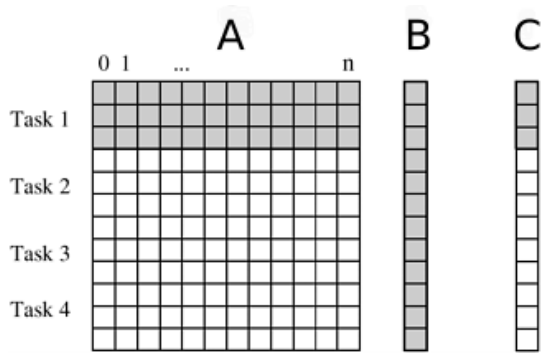

Figure 2: http://parallelcomp.uw.hu/ch03lev1sec1.html

Our the parallelization scheme is the same as shown here. Each task sees one or more rows of A, all of B, and calculates a subset of C.

3 Results

The results presented in this section were calculated by

1. Learn MPI ICC Compiler.
2. Learn Intel profiler (MPI Trace Output?).
3. recalculate communication and computation times.
4. ask prof if graphs look good.

3.1 Total Matrix Calculation Time

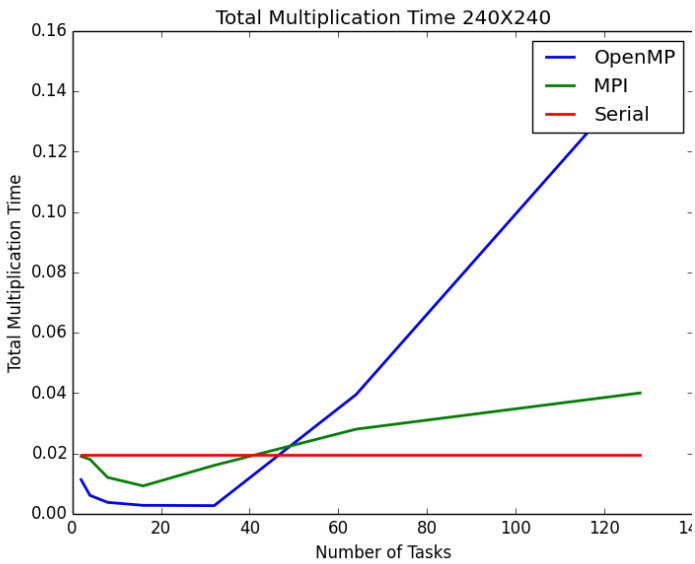

Figure 3: A figure

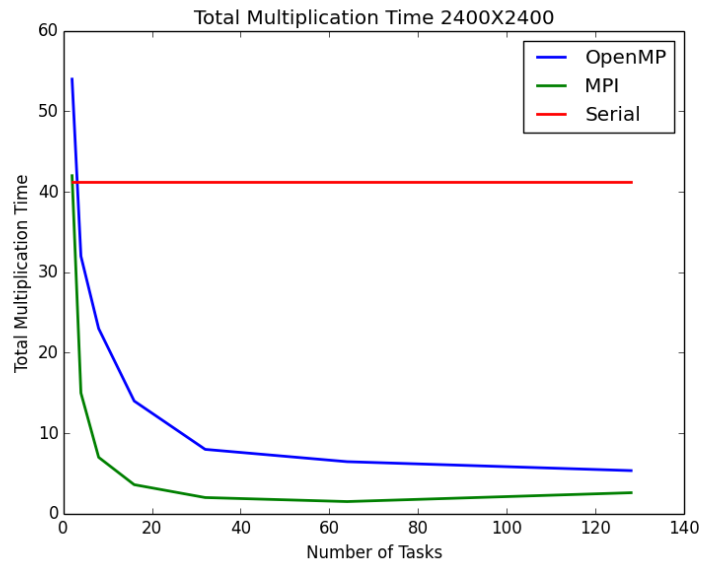

Figure 4: Another figure

Amdahl's Law states

$$speedup = 1/(P/N + S) \tag{2}$$

where P = parallel fraction, N = number of processors and S = serial fraction [lecture 2 notes].

$$totaltime = time0/speedup = time0 \times (P/N + S) \tag{3}$$

??? Law [lecture 2] the time (t) required to send W words between any two processors is:

$$t = L + W/B \tag{4}$$

where L = latency and B = hardware bitstream rate in words per second.

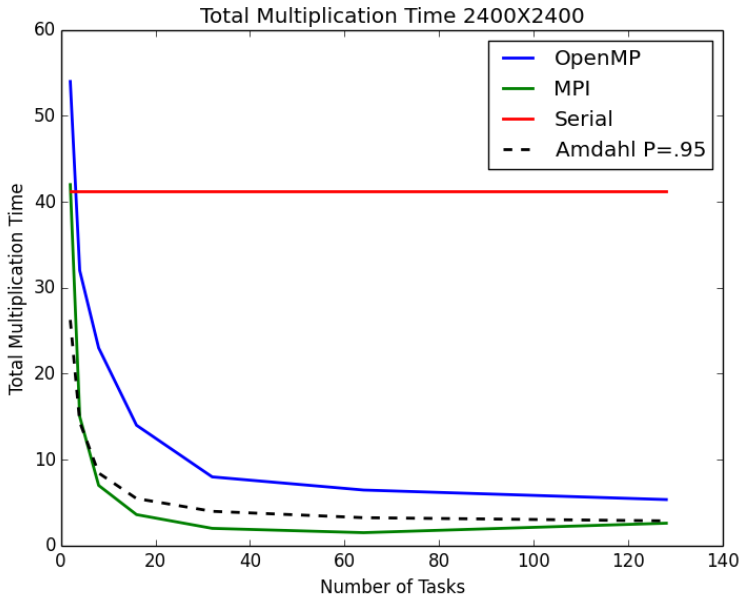

Figure 5: todo

3.2 Ratio of Communication Over Calculation Time

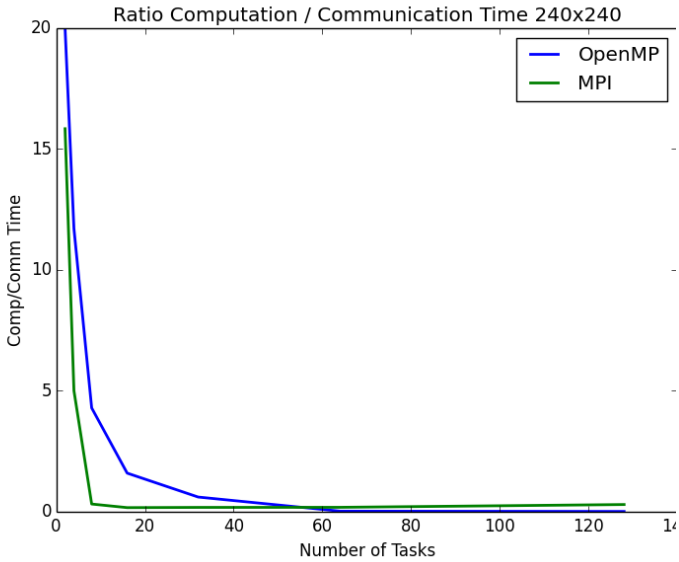

Figure 6: A figure

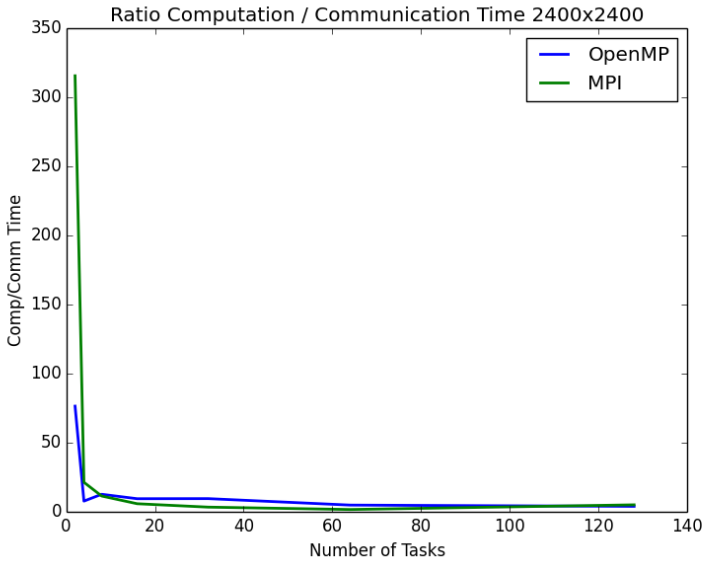

Figure 7: Another figure

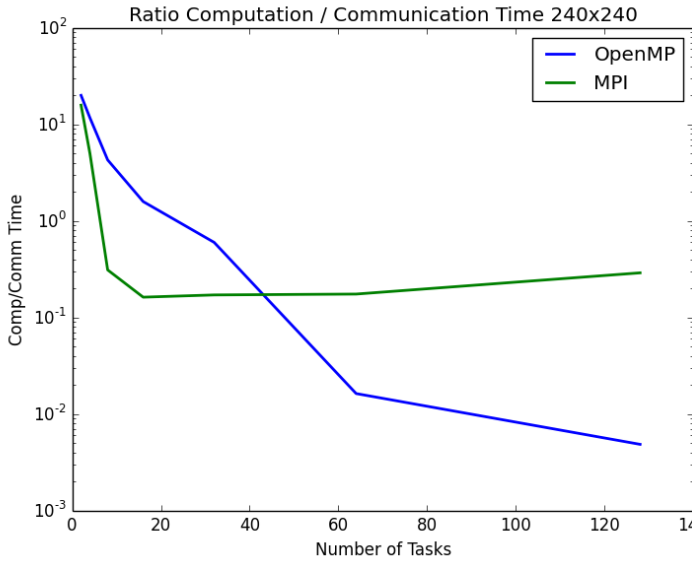

Figure 8: A figure

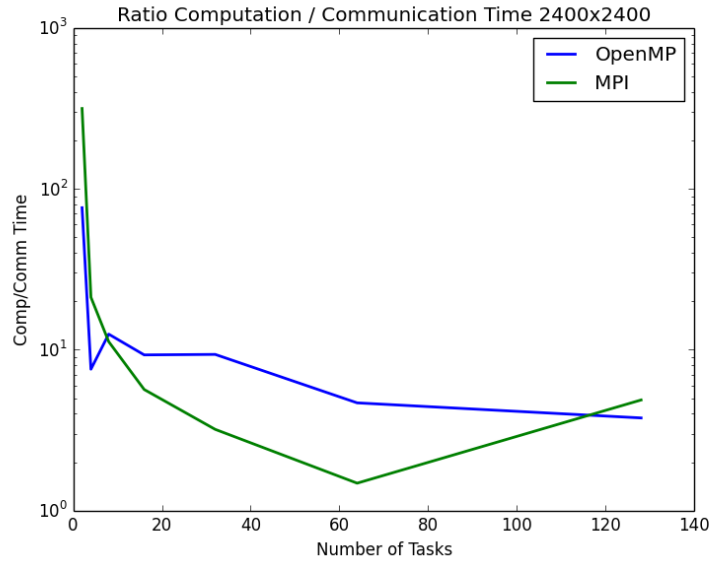

Figure 9: Another figure

Appendices

A Serial Code

```
#include<stdio.h>
#include<cstdlib>
#include<time.h>
#include<iostream>
#include<matrix.h>

/* Function Prototypes */
void FillMatricesRandomly(Matrix<double> &A, Matrix<double> &B );
void PrintMatrices( Matrix<double> &A, Matrix<double> &B, Matrix<double> &C );

/* Global variables that could be inputs #TODO */
int randomHigh = 100;           /* the upper bound of the random numbers that fill A and B*/
int randomLow = 0;              /* the lower bound of the random numbers that fill A and B*/

int main(int argc, char *argv[]) {
    std::cout << "Starting a serial matrix multiplication. \n  " << std::endl;
  // Read in the input:  N
  if ( argv[1]== NULL ){ // check input is supplied
    std::cout << "ERROR: The program must be executed in the following way  \n\n  \t \"./serial.exe N \"  \n\n where N is an integer. \n \n " << std::endl;
    return 1;
  }
  int N = atoi(argv[1]);  // The dimensions of the matrices MUST be specified at runtime.
  std::cout << "The matrices are: " << N<<"x"<<N<< std::endl;
  // for simplicity, all matricies will be NxN
  int numberOfRowsA = N;  int numberOfColsA = N;  int numberOfRowsB = N;  int numberOfColsB = N;

  //Declare matrices: Matix class is a 2D vetor
  Matrix<double> A = Matrix<double>(numberOfRowsA, numberOfColsA);
  Matrix<double> B = Matrix<double>(numberOfRowsB, numberOfColsB);
  Matrix<double> C = Matrix<double>(numberOfRowsA, numberOfColsB);

  FillMatricesRandomly(A, B);   /* Excluded from timing!!!  */

  struct timespec start, end;
  clock_gettime(CLOCK_MONOTONIC, &start); // start timing

  // Matrix Multiplication
  for (int i = 0; i < A.rows(); i++) {//iterate through rows of A
```

```
    for (int j = 0; j < B.cols(); j++) {//iterate through columns of B
      for (int k = 0; k < B.rows(); k++) {//iterate through rows of B
        C(i,j) += (A(i,k) * B(k,j));
      }
    }
  }

  clock_gettime(CLOCK_MONOTONIC, &end); // end timing
  double matrixCalculationTime = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec)*1e-9;
  std::cout << "\nTotal multplication time = " << matrixCalculationTime << std::endl;
//   PrintMatrices(A, B, C);
  return 0;
}

void FillMatricesRandomly(Matrix<double> &A, Matrix<double> &B){
/* initialize the random number generator with the current time */
  srand( time( NULL ));
  for (int i = 0; i < A.rows(); i++) {
    for (int j = 0; j < A.cols(); j++) {
      A(i,j) = rand() % (randomHigh - randomLow) + randomLow;
    }
  }
  for (int i = 0; i < B.rows(); i++) {
    for (int j = 0; j < B.cols(); j++) {
      B(i,j) = rand() % (randomHigh - randomLow) + randomLow;
    }
  }
}


void PrintMatrices(Matrix<double> &A, Matrix<double> &B, Matrix<double> &C){
  for (int i = 0; i < A.rows(); i++) {
    std::cout <<"\n"<<std::endl;
    for (int j = 0; j < A.cols(); j++)
      std::cout << A(i,j) << " ";
  }
    std::cout <<"\n\n"<<std::endl;
  for (int i = 0; i < B.rows(); i++) {
    std::cout <<"\n"<<std::endl;
    for (int j = 0; j < B.cols(); j++)
      std::cout << B(i,j) << " ";
  }
  std::cout <<"\n\n"<<std::endl;
  for (int i = 0; i < C.rows(); i++) {
    std::cout <<"\n"<<std::endl;
    for (int j = 0; j < C.cols(); j++)
      std::cout << C(i,j) << " ";
  }
  std::cout <<"\n\n"<<std::endl;
}
```

B OpenMP Code

```
#include<cstdlib>
#include<time.h>
#include<iostream>
#include<matrix.h>
#include<omp.h>


/* Function Prototypes */
void FillMatricesRandomly(Matrix<double> &A, Matrix<double> &B );
void PrintMatrices( Matrix<double> &A, Matrix<double> &B, Matrix<double> &C );

/* Global variables that could be inputs #TODO */
int randomHigh = 100;           /* the upper bound of the random numbers that fill A and B*/
int randomLow = 0;              /* the lower bound of the random numbers that fill A and B*/

int main(int argc, char *argv[]) {
    std::cout << "Starting an OpenMP parallel matrix multiplication. \n  " << std::endl;
  // Read in the two inputs: NumberOfOMPthreads and N
  if ( argv[1]== NULL || argv[2] == NULL){ // check if inputs were supplied
    std::cout << "ERROR: The program must be executed in the following way  \n\n  \t \"./omp.exe NumberOfThreads N \"  \n\n where NuberOfThreads and N are integers. \n \n " << std::endl;
    return 1;
  }
  int numThreads = atoi(argv[1]);
  std::cout << "The number of OpenMP threads: " << numThreads << std::endl;
  omp_set_dynamic(0);           // do not allow the number of threads to be set internally
  omp_set_num_threads(numThreads); // set the number of threads

  int N = atoi(argv[2]);  // The dimensions of the matrices MUST be specified at runtime.
  std::cout << "The matrices are: " << N<<"x"<<N<< std::endl;
  // for simplicity, all matricies will be NxN
  int numberOfRowsA = N;  int numberOfColsA = N;  int numberOfRowsB = N;  int numberOfColsB = N;
  //Declare matrices: Matix class is a 2D vetor
  Matrix<double> A = Matrix<double>(numberOfRowsA, numberOfColsA);
  Matrix<double> B = Matrix<double>(numberOfRowsB, numberOfColsB);
  Matrix<double> C = Matrix<double>(numberOfRowsA, numberOfColsB);

  FillMatricesRandomly(A, B);   /* Excluded from timing!!!  */

  struct timespec start, end;
  clock_gettime(CLOCK_MONOTONIC, &start); // start timing

  // Used to calculate the longest a process spends calculating its part of the workload.
  // double sumLocalTime = 0;
  double sum = 0;
  double val = 0;
  // Matrix Multiplication
#pragma omp parallel for private(val) reduction(+:sum)
  for (int i = 0; i < A.rows(); i++) {//iterate through rows of A (parallelized loop)
    val = omp_get_wtime();
    for (int j = 0; j < B.cols(); j++) {//iterate through columns of B
      for (int k = 0; k < B.rows(); k++) {//iterate through rows of B
        C(i,j) += (A(i,k) * B(k,j));
      }
    }
    sum +=  omp_get_wtime() - val;
  }

  clock_gettime(CLOCK_MONOTONIC, &end); // end timing
  double totalMatrixCalculationTime = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) *1e-9;
  std::cout << "Total multplication time = " << totalMatrixCalculationTime << std::endl;
  std::cout << "average multplication time = " << sum/numThreads << std::endl;
  std::cout << "Approximate Communication time = " << totalMatrixCalculationTime - sum/numThreads << std::endl;
//   PrintMatrices(A, B, C);
  return 0;
}

void FillMatricesRandomly(Matrix<double> &A, Matrix<double> &B){
/* initialize the random number generator with the current time */
```

```
  srand( time( NULL ));
  for (int i = 0; i < A.rows(); i++) {
    for (int j = 0; j < A.cols(); j++) {
      A(i,j) = rand() % (randomHigh - randomLow) + randomLow;
    }
  }
  for (int i = 0; i < B.rows(); i++) {
    for (int j = 0; j < B.cols(); j++) {
      B(i,j) = rand() % (randomHigh - randomLow) + randomLow;
    }
  }
}

void PrintMatrices(Matrix<double> &A, Matrix<double> &B, Matrix<double> &C){
  for (int i = 0; i < A.rows(); i++) {
    std::cout <<"\n"<<std::endl;
    for (int j = 0; j < A.cols(); j++)
      std::cout << A(i,j) << " ";
  }
    std::cout <<"\n\n"<<std::endl;
  for (int i = 0; i < B.rows(); i++) {
    std::cout <<"\n"<<std::endl;
    for (int j = 0; j < B.cols(); j++)
      std::cout << B(i,j) << " ";
  }
  std::cout <<"\n\n"<<std::endl;
  for (int i = 0; i < C.rows(); i++) {
    std::cout <<"\n"<<std::endl;
    for (int j = 0; j < C.cols(); j++)
      std::cout << C(i,j) << " ";
  }
  std::cout <<"\n\n"<<std::endl;

}
```

C MPI Code

```
#include<cstdlib>
#include<ctime>
#include<mpi.h>
#include<algorithm>
#include<matrix.h>

// Function Prototypes
void FillMatricesRandomly(Matrix<double> &A, Matrix<double> &B );
void PrintMatrices( Matrix<double> &A, Matrix<double> &B, Matrix<double> &C );

/* MPI Send and Recieve Tags */
#define ROW_START_TAG 0 //tag for communicating the start row of the workload for a slave
#define ROW_END_TAG 1   //tag for communicating the end row of the workload for a slave
#define A_ROWS_TAG 2    //tag for communicating the address of the data to be worked on to slave
#define C_ROWS_TAG 3    //tag for communicating the address of the calculated data to master
#define LOCAL_TIME_TAG 4//tag for communicating the address of the local matrix calculation time to master

// Instantiate global variables used in the parallelization
int rank;                   // mpi: process id number
int nProcesses;             // mpi: number of total processess
MPI_Status status;          // mpi: store status of a MPI_Recv
MPI_Request request;        // mpi: capture request of a MPI_Isend
int rowStart, rowEnd;       // which rows of A that are calculated by the slave process
int granularity;            // granularity of parallelization (# of rows per processor)

//Used to calculate totalmultiplication time: communication + calculations
double start_time, end_time;
// Used to calculate the longest a process spends calculating its part of the workload.
double localTimeSaver;

/* Global variables that could be inputs #TODO */
int randomHigh = 100;     // the upper bound of the random numbers that fill A and B
int randomLow = 0;        // the lower bound of the random numbers that fill A and B

int main(int argc, char *argv[]) {
  // ***** Handle the input size of the Matrices, N, where matrices A, B, and C will be NxN *****
  if ( argv[1]== NULL ){// check if the input was supplied
   std::cout << "ERROR: The program must be executed in the following way  \n\n  \t \"mpirun -n NumberOfProcesses mpi.exe N \"  \n\n where N is an integer. \n \n " << std::endl;
   return 1;
  }
  int N = atoi(argv[1]);  // The dimensions of the matrices MUST be specified at runtime.
  // for simplicity, all matricies will be NxN
  int numberOfRowsA = N;  int numberOfColsA = N;  int numberOfRowsB = N;  int numberOfColsB = N;
  //Declare matrices: Matix class is a 2D vetor
  Matrix<double> A = Matrix<double>(numberOfRowsA, numberOfColsA);
  Matrix<double> B = Matrix<double>(numberOfRowsB, numberOfColsB);
  Matrix<double> C = Matrix<double>(numberOfRowsA, numberOfColsB);

  // MPI:
  MPI_Init(&argc, &argv);                      /* initialize MPI */
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);        /* store the rank */
  MPI_Comm_size(MPI_COMM_WORLD, &nProcesses); /* store the number of processes */

  if (rank == 0) {  /* Master initializes work*/
    std::cout << "Starting an MPI parallel matrix multiplication. \n  " << std::endl;
    std::cout << "The matrices are: " << N<<"x"<<N<< std::endl;
    FillMatricesRandomly( A, B);        /* Excluded from timing!!!  */

    /* Begin Timing: used for total multiplication time: communication + calculations  */
    start_time = MPI_Wtime();
    for (int i = 1; i < nProcesses; i++) { /* for each slave */
      // calculate granularity (-1 comes from excluding the master process)
      granularity = (numberOfRowsA / (nProcesses - 1));
      rowStart = (i - 1) * granularity;
      if (((i + 1) == nProcesses) && ((numberOfRowsA % (nProcesses - 1)) != 0)) {//if rows of [A] cannot be equally divided among slaves
        rowEnd = numberOfRowsA; //last slave gets all the remaining rows
      } else {
        rowEnd = rowStart + granularity; //rows of [A] are equally divisable among slaves
```

```
      }
      //send the low bound, without blocking, to the intended slave
      MPI_Isend(&rowStart, 1, MPI_INT, i, ROW_END_TAG, MPI_COMM_WORLD, &request);
      //next send the upper bound without blocking, to the intended slave
      MPI_Isend(&rowEnd, 1, MPI_INT, i ,ROW_START_TAG, MPI_COMM_WORLD, &request);
      //finally send the allocated row granularity of [A] without blocking, to the intended slave
      MPI_Isend(&A(rowStart,0), (rowEnd - rowStart) * numberOfColsA, MPI_DOUBLE, i, A_ROWS_TAG, MPI_COMM_WORLD, &request);
    }
  }
  //broadcast B (MPI_Bcast: Broadcasts a message from the process with rank "root" to all other processes of the communicator)
  MPI_Bcast(&B(0,0), numberOfRowsB*numberOfColsB, MPI_DOUBLE, 0, MPI_COMM_WORLD);

  if (rank > 0) {   /* work done by slaves (not rank = 0)*/
    //receive low bound from the master
    MPI_Recv(&rowStart, 1, MPI_INT, 0, ROW_END_TAG, MPI_COMM_WORLD, &status);
    //next receive upper bound from the master
    MPI_Recv(&rowEnd, 1, MPI_INT, 0, ROW_START_TAG, MPI_COMM_WORLD, &status);
    //finally receive row granularity of [A] to be processed from the master
    MPI_Recv(&A(rowStart,0), (rowEnd - rowStart) * numberOfColsA, MPI_DOUBLE, 0, A_ROWS_TAG, MPI_COMM_WORLD, &status);

    // start time for local time: the amount of time to do matrix calculation for this process
    localTimeSaver = MPI_Wtime();
    /* Matrix Multiplication  */
    for (int i = rowStart; i < rowEnd; i++) {//the given set of rows of A (parallelized loop)
      for (int j = 0; j < B.cols(); j++) {//iterate through columns of [B]
        for (int k = 0; k < B.rows(); k++) {//iterate through rows of [B]
          C(i,j) += (A(i,k) * B(k,j));
        }
      }
    }
    // calculate local time: the amount of time to do matrix calculation for this process
    localTimeSaver = MPI_Wtime() - localTimeSaver;

    //send back the low bound first without blocking, to the master
    MPI_Isend(&rowStart, 1, MPI_INT, 0, ROW_END_TAG, MPI_COMM_WORLD, &request);
    //send the upper bound next without blocking, to the master
    MPI_Isend(&rowEnd, 1, MPI_INT, 0, ROW_START_TAG, MPI_COMM_WORLD, &request);
    //finally send the processed granularity of data without blocking, to the master
    MPI_Isend(&C(rowStart,0), (rowEnd - rowStart) * numberOfColsB, MPI_DOUBLE, 0, C_ROWS_TAG, MPI_COMM_WORLD, &request);
    //send back the local calculation time without blocking, to the master
    MPI_Isend(&localTimeSaver, 1, MPI_DOUBLE, 0, LOCAL_TIME_TAG, MPI_COMM_WORLD, &request);


  }

  if (rank == 0) { /* master gathers processed work*/
    for (int i = 1; i < nProcesses; i++) {// untill all slaves have handed back the processed data
      //receive low bound from a slave
      MPI_Recv(&rowStart, 1, MPI_INT, i, ROW_END_TAG, MPI_COMM_WORLD, &status);
      //receive upper bound from a slave
      MPI_Recv(&rowEnd, 1, MPI_INT, i, ROW_START_TAG, MPI_COMM_WORLD, &status);
      // //receive processed data from a slave
      MPI_Recv(&C(rowStart,0), (rowEnd - rowStart) * numberOfColsB, MPI_DOUBLE, i, C_ROWS_TAG, MPI_COMM_WORLD, &status);

    }
    end_time = MPI_Wtime(); //end time of the total matrix matrix multiplication
    double totalMultiplicationTime = end_time - start_time;

    // find the longest local calulation (which we take as the total amount of calculation time, the rest comming from communication.
    std::vector<double> LocalMultiplicationTimes = std::vector<double>(nProcesses);
    for (int i = 1; i < nProcesses; i++) {
      MPI_Recv(&LocalMultiplicationTimes[i], 1, MPI_DOUBLE, i, LOCAL_TIME_TAG, MPI_COMM_WORLD, &status);
    }
    double maxLocalMultiplicationTime = *std::max_element(LocalMultiplicationTimes.begin(), LocalMultiplicationTimes.end());

```

```
    // print out the results
    std::cout <<"Total multiplication time =  " << totalMultiplicationTime <<"\n"<< std::endl;
    std::cout <<"Longest multiplication time =  " << maxLocalMultiplicationTime  <<"\n"<< std::endl;
    std::cout <<"Approximate communication time =  " << totalMultiplicationTime - maxLocalMultiplicationTime  <<"\n\n"<< std::endl;

    // PrintMatrices(A, B, C);     // for debugging

  }
  MPI_Finalize(); //finalize MPI operations
  return 0;
}

void FillMatricesRandomly(Matrix<double> &A, Matrix<double> &B){
/* initialize the random number generator with the current time */
  srand( time( NULL ));
  for (int i = 0; i < A.rows(); i++) {
    for (int j = 0; j < A.cols(); j++) {
      A(i,j) = rand() % (randomHigh - randomLow) + randomLow;
    }
  }
  for (int i = 0; i < B.rows(); i++) {
    for (int j = 0; j < B.cols(); j++) {
      B(i,j) = rand() % (randomHigh - randomLow) + randomLow;
    }
  }
}


void PrintMatrices(Matrix<double> &A, Matrix<double> &B, Matrix<double> &C){
  for (int i = 0; i < A.rows(); i++) {
    std::cout <<"\n"<<std::endl;
    for (int j = 0; j < A.cols(); j++)
      std::cout << A(i,j) << " ";
  }
    std::cout <<"\n\n"<<std::endl;
  for (int i = 0; i < B.rows(); i++) {
    std::cout <<"\n"<<std::endl;
    for (int j = 0; j < B.cols(); j++)
      std::cout << B(i,j) << " ";
  }
  std::cout <<"\n\n"<<std::endl;
  for (int i = 0; i < C.rows(); i++) {
    std::cout <<"\n"<<std::endl;
    for (int j = 0; j < C.cols(); j++)
      std::cout << C(i,j) << " ";
  }
  std::cout <<"\n\n"<<std::endl;
}
```

D Matrix.h

```
#ifndef MATRIX_H_
#define MATRIX_H_

#include <vector>
#include <complex>

using std::vector;
using std::complex;

// simple wrapper of a vector
template <class T>
class Matrix {
public:
  // create an empty matrix
  Matrix(int numrows, int numcols)
    :Nrow(numrows), Ncol(numcols), elements(Nrow*Ncol) {}

  // construct it from existing data
  Matrix(int numrows, int numcols, T* data)
    :Nrow(numrows), Ncol(numcols), elements(data, data+numrows*numcols) {}

  int rows() {return Nrow;}
  int cols() {return Ncol;}

  // access to elements, col is the fast axis
  T operator() (int row, int col) const {return elements[Ncol*row + col];}
  T& operator() (int row, int col) {return elements[Ncol*row + col];}


  // get raw pointer to elements[0]
  T* data() {return elements.data();}
  const vector<T>& elem() {return elements;}

private:
  int Nrow, Ncol;
  vector<T> elements;
};


#endif // MATRIX_H_
```